EXPERIMENT 10

Performance Optimization in Zoho Creator

Efficient workflow processing in the Gym Membership Management System

Submitted by	PRANAV SRINEVAS V	**Register No.**	241401068
Application	Gym_Membership_Management_System (Zoho Creator)	**Department**	B.Tech CSBS

1. Aim

To study performance optimization in Zoho Creator by examining the member-registration workflow of the gym application, which reads only the records it needs, and the bulk CSV import used to load the members.

2. Introduction

Performance optimization in a low-code application means writing forms, workflows and scripts that use data and processing only as far as needed. Fetching more records than required, or processing records that are not relevant, wastes effort. This is not noticeable in a small application, but it reduces efficiency as records and users grow.

3. Application used

The practical was done on the Gym_Membership_Management_System application. A Deluge script in a form workflow on Member_Registration runs on Created → Successful form submission and sends the welcome e-mail.

4. Performance optimization concept

Criteria-based fetching means retrieving only the record needed by giving a criteria, for example `Membership_Plans[ID == input.Membership_Plans]`, instead of fetching every record and looping through them. Because the workflow runs on the form's own submission, the values of the new member are already available through `input`.

```
// Inefficient: fetch every plan, then search for the chosen one
plans = Membership_Plans[];
for each p in plans
{
    if(p.ID == input.Membership_Plans)
    {
        // ... process
    }
}

// Used in the gym workflow: one criteria-based fetch by ID
planRec = Membership_Plans[ID == input.Membership_Plans];
```

Approach	Steps
Inefficient	Fetch all plan records, iterate, find the chosen plan, process.
Used in the gym workflow	Fetch only the chosen plan by ID; read the member's own values from input.

5. Procedure

1. Open Gym_Membership_Management_System in Edit mode and go to Workflow → Form workflows.
2. Open the member-registration workflow; it runs on Created → Successful form submission and has a Deluge script action.
3. Read the script (Code 1): it fetches the plan once by ID, fetches the member record once by ID, handles optional values with checks, builds the message once and sends one e-mail.
4. Register a member and check that the welcome e-mail arrives.
5. Load the member data with the CSV import instead of typing each member.

[Screenshot to be inserted]
Insert the Form workflows list screenshot and the Deluge script screenshot.

Code 1: Gym workflow script (excerpt)

```
// 1. Get plan details from the Membership_Plans lookup
planName = "";
planFee = "";
duration = 0;
planRec = Membership_Plans[ID == input.Membership_Plans];
if(planRec.count() > 0)
{
    planName = planRec.Plan_Name;
    planFee = planRec.Plan_Fee;
    // Duration worked out from the plan name
    pn = planName.toString().toLowerCase();
    if(pn.contains("basic"))
    {
        duration = 1;
    }
    else if(pn.contains("standard"))
    {
        duration = 3;
    }
    else if(pn.contains("gold"))
    {
        duration = 6;
    }
    else if(pn.contains("platinum"))
    {
        duration = 9;
    }
    else if(pn.contains("diamond"))
    {
        duration = 12;
    ...
```

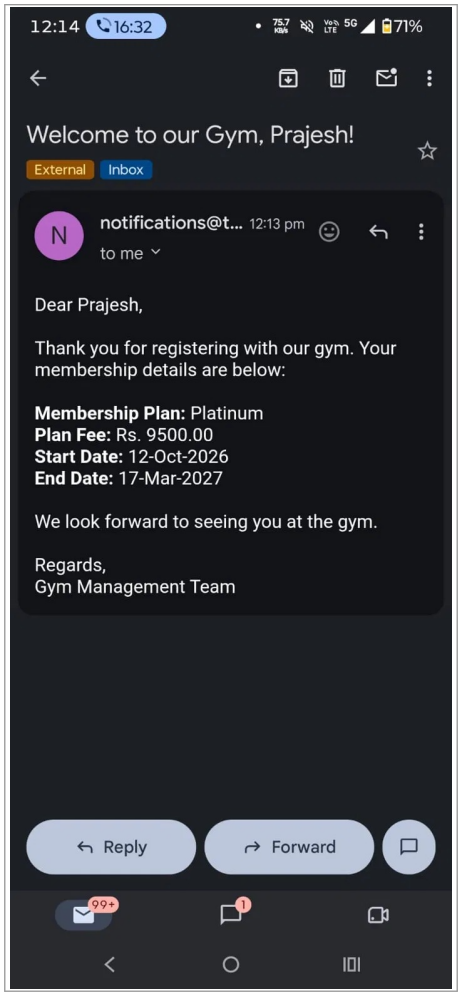


Figure 1: Confirmation e-mail received after the workflow ran.

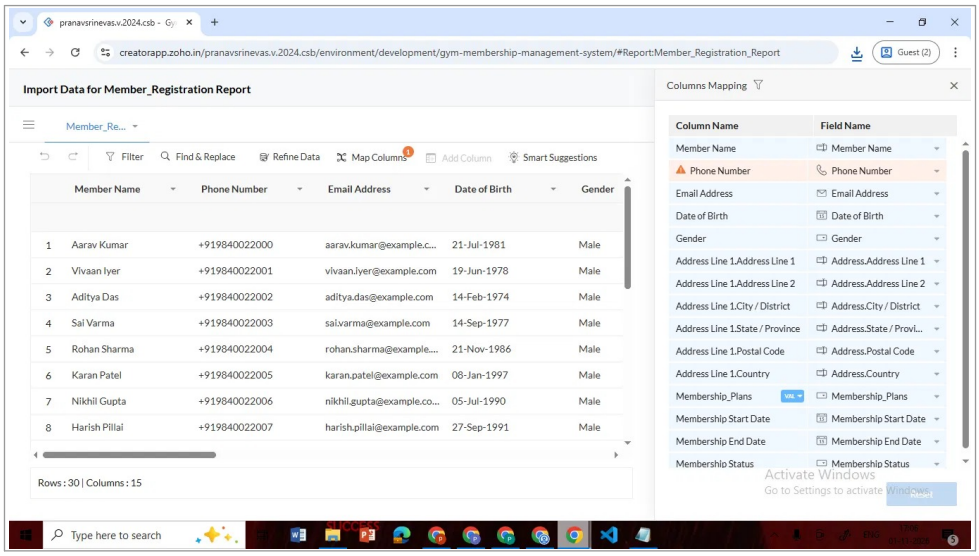


Figure 2: Bulk CSV import of the gym members (30 rows, 15 columns) in a single operation.

6. Analysis of the script

Part of the script	Why it is efficient
`Membership_Plans[ID == input.Membership_Plans]`	One record is fetched by its ID; the other four plans are not read.
`Member_Registration[ID == input.ID]`	Only the submitted member is read and updated.

Part of the script	Why it is efficient
`if(planRec.count() > 0)` and null checks	Missing plans or dates are handled without extra lookups.
E-mail check before sending	Mail is sent only when an address exists.
Message built once, one sendmail	A single e-mail is composed and sent; there are no loops or repeated sends.

7. Optimization techniques

Technique	Explanation	Status in this practical
Criteria-based record fetching	Retrieve only the records required.	Used (plan and member fetched by ID).
Avoiding unnecessary loops	Do not fetch all records and loop when a criteria can be used.	Followed: the script has no loops.
Bulk import	Load many records in one operation.	Used: 30 members in one CSV import.
Built-in functions	Use addMonth(), toString() and contains() instead of manual processing.	Used in the script.
Indexed fields	May speed retrieval of frequently queried data.	Not configured.

8. Result and Conclusion

The gym workflow reads only the records it needs, has no loops and sends a single e-mail, and the member data was loaded with one import instead of 30 manual entries. No benchmark was run, so no measured improvement in execution time is claimed; the benefit is described in terms of data retrieved and processed. As the gym grows, these practices keep the application efficient and maintainable.